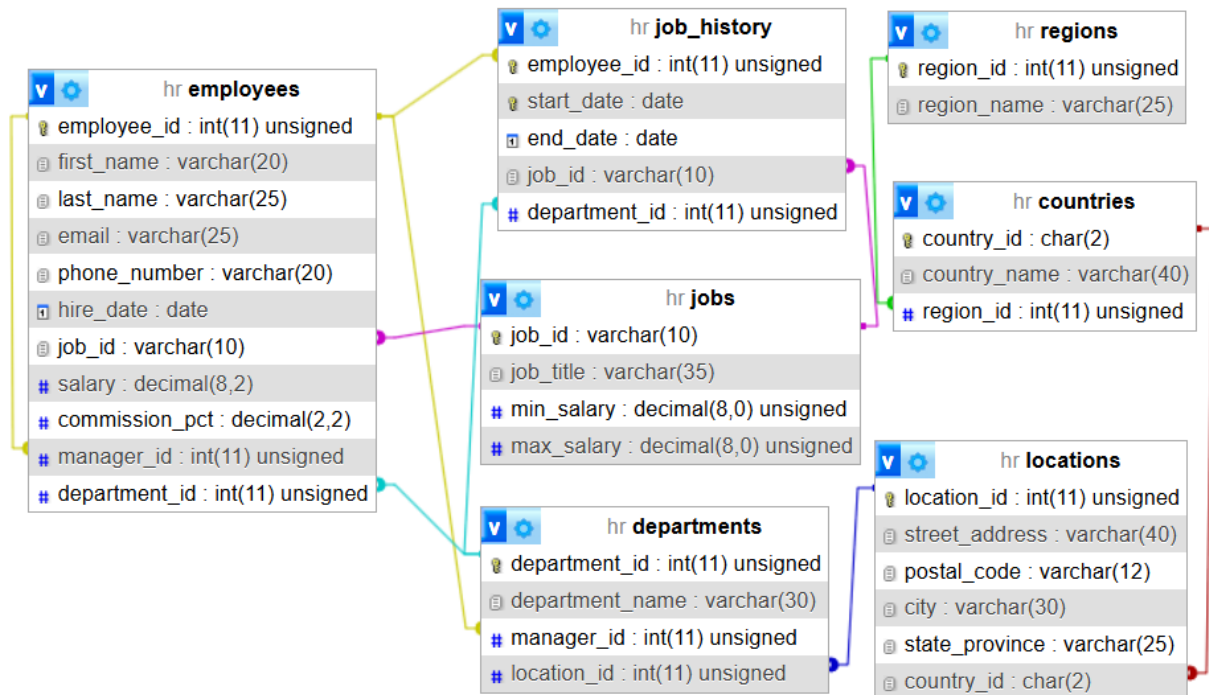


Database Management System Laboratory (CSE 3522)

WEEK 6

Topic: JOIN Operation, SUBQUERY



Database Source: https://github.com/enamulrafi/dbms_lab_contents/blob/main/hr-schema-mysql.sql

❖ JOIN OPERATION

A JOIN is an SQL operation used to combine rows from two or more tables based on a related column between them. Instead of storing all data in one table, relational databases keep data in multiple tables. JOIN helps query data across tables efficiently. There are many types of JOIN operation, such as INNER JOIN, LEFT JOIN, RIGHT JOIN, FULL JOIN, CROSS JOIN, SELF JOIN

- 1) **INNER JOIN/JOIN**: INNER JOIN/JOIN returns only the rows that have matching values in both tables. It filters out rows from either table that don't have a corresponding match in the other table.

Query Questions:

Q1: List all employees along with their department names.

```
SELECT e.first_name, e.last_name, d.department_name
FROM employees e
JOIN departments d ON e.department_id = d.department_id;
```

Q2: Show employees with their job titles and salary ranges.

```
SELECT e.first_name, e.last_name, e.salary, j.job_title, j.min_salary, j.max_salary
FROM employees e
INNER JOIN jobs j ON e.job_id = j.job_id;
```

Q3: Find employees who have worked in multiple departments (using job_history).

```
SELECT e.first_name, e.last_name, jh.start_date, jh.end_date, d.department_name
FROM employees e
INNER JOIN job_history jh ON e.employee_id = jh.employee_id
INNER JOIN departments d ON jh.department_id = d.department_id;
```

- 2) **LEFT JOIN (LEFT OUTER JOIN)**: LEFT JOIN returns all rows from the left table (first table), and the matched rows from the right table. If no match exists, NULL values are returned for columns from the right table.

Query Questions:

Q1: List all employees and their departments, including employees without departments.

```

SELECT e.first_name, e.last_name, d.department_name
FROM employees e
LEFT JOIN departments d ON e.department_id = d.department_id
WHERE d.department_name IS NOT NULL;

```

Q2: Find all departments and their managers, including departments without managers.

```

SELECT d.department_name, e.first_name AS manager_first, e.last_name AS manager_last
FROM departments d
LEFT JOIN employees e ON d.manager_id = e.employee_id
ORDER BY d.department_name;

```

Q3: Show all locations with their countries, including locations without country info.

```

SELECT l.city, l.state_province, c.country_name
FROM locations l
LEFT JOIN countries c ON l.country_id = c.country_id;

```

- 3) **RIGHT JOIN (RIGHT OUTER JOIN):** RIGHT JOIN returns all rows from the right table, and the matched rows from the left table. If no match exists, NULL values are returned for columns from the left table.

Query Questions:

Q1: List all departments and their employees, including empty departments.

```

SELECT d.department_name, e.first_name, e.last_name
FROM employees e
RIGHT JOIN departments d ON e.department_id = d.department_id
ORDER BY d.department_name, e.last_name;

```

Q2: Show all countries and their locations, including countries without locations.

```

SELECT c.country_name, l.city, l.state_province
FROM locations l
RIGHT JOIN countries c ON l.country_id = c.country_id
ORDER BY c.country_name;

```

- 4) **FULL OUTER JOIN:** FULL OUTER JOIN returns all rows when there is a match in either left or right table. It combines the results of both LEFT and RIGHT joins. If there's no match, NULL values are returned for missing sides.

Note: MySQL doesn't natively support FULL OUTER JOIN, so we simulate it with UNION of LEFT and RIGHT JOIN.

❖ Subquery

A subquery is a query written inside another SQL query. The inner query is executed first, and its result is then used by the outer (main) query. In simple words, a query inside another query. A subquery is usually placed inside: WHERE, HAVING, FROM, SELECT.

There are different types of subqueries. 1. Scaler subquery 2. Column Subquery 3. Row Subquery 4. Derived Table 5. Correlated subquery

- 1) **SCALAR SUBQUERY:** A subquery that returns exactly one value (one row and one column). It can be used anywhere a single value (constant) is expected, such as in a SELECT list or a conditional expression.

Q1: Find employees earning more than the company's average salary.

-- 1. Find company average salary: `SELECT AVG(salary) FROM employees`

-- 2. Compare each employee's salary > that average

```

SELECT employee_id, first_name, last_name, salary
FROM employees
WHERE salary > (
    SELECT AVG(salary)
    FROM employees );

```

Q2: Show each employee with their department's highest salary.

-- For each employee, find their department's max salary

```
-- Compare employee salary = their department's max
SELECT e.first_name, e.last_name, e.salary, d.department_name
FROM employees e
JOIN departments d ON e.department_id = d.department_id
WHERE e.salary = (
    SELECT MAX(salary)
    FROM employees
    WHERE department_id = e.department_id -- Scalar for each department
);
```

Q3: Show employee salary with difference of their company average salary.

- 1. For EACH employee, show their details
- 2. Also show the company's average salary (same for all rows), and salary difference from the dept.

```
SELECT employee_id, first_name, last_name, salary,
    (SELECT AVG(salary) FROM employees) AS company_avg_salary,
    salary - (SELECT AVG(salary) FROM employees) AS diff_from_avg
FROM employees
ORDER BY diff_from_avg DESC;
```

- 2) **COLUMN SUBQUERY:** A column subquery in MySQL is a subquery that returns one column (multiple row) to the main query. It is typically used in the SELECT clause to include calculated or derived values for each row. Similar to scalar subquery with extra operator: ANY, ALL, IN

Query Questions:

Q1: Find employees who work in departments located in the United States (US).

```
-- 1. Find location_ids in US
-- 2. Find department_ids in those locations
-- 3. Find employees in those departments
SELECT e.employee_id, e.first_name, e.last_name, d.department_name
FROM employees e
JOIN departments d ON e.department_id = d.department_id
WHERE e.department_id IN (
    SELECT department_id
    FROM departments
    WHERE location_id IN (
        SELECT location_id
        FROM locations
        WHERE country_id = 'US'
    )
);
```

- 3) **ROW SUBQUERY:** Returns one row with multiple columns. Used for multi-column comparisons. Use when comparing multiple columns simultaneously

Query Questions:

Q1: Find employees with the same job and department as employee id 120.

```
-- 1. Get job_id and department_id of employee 120: (job_id, department_id)
-- 2. Find employees with same (job_id, department_id) pair
SELECT employee_id, first_name, last_name, job_id, department_id
FROM employees
WHERE (job_id, department_id) = (
    SELECT job_id, department_id FROM employees WHERE employee_id = 120
);
```

Q2: Find the highest paid employee-job combination in each department.

-- For each department, find max salary for each job

-- Compare employee (salary, job_id) = department's max per job

```
SELECT e.first_name, e.last_name, e.job_id, e.salary, d.department_name
FROM employees e
JOIN departments d ON e.department_id = d.department_id
WHERE (e.salary, e.job_id) IN (
    SELECT MAX(salary), job_id
    FROM employees e2
    GROUP BY job_id
);
```

Q3: Find employees hired on the earliest date in their department.

-- For each department, find earliest hire_date

-- Compare employee (hire_date, department_id) pair

```
SELECT e.first_name, e.last_name, e.hire_date, d.department_name
FROM employees e
JOIN departments d ON e.department_id = d.department_id
WHERE (e.hire_date, e.department_id) IN (
    SELECT MIN(hire_date), department_id
    FROM employees
    GROUP BY department_id
);
```